Learn
Practice
Build
Grow !

Day 1 : Version Control Basics

"Track Changes · Collaborate Better · Build Great Software !"

Small
Commits
Big Progress !

Version
Control
= Peace of Mind

1 What is Version Control ?

- Version control is a system that records changes to files over time.
- It helps you track history, revert to previous versions and collaborate with others.
- Widely used for source code, configuration files, documentation, etc.
- Allows multiple people to work on the same project without overwriting each other's changes.

"It's like a time machine for your code !"

2 Centralized vs Distributed VCS

Feature	Centralized VCS	Distributed VCS (Git)
Architecture	Single central server	Every user has a full copy (local repo)
Example	SVN, CVS	Git, Mercurial
Offline work	Not possible	Possible (full history locally)
Speed	Slower (network dependency)	Faster (local operations)
Single point of failure	Yes	No
Branching	Limited	Easy and powerful
Popular use	Older setups	Modern development

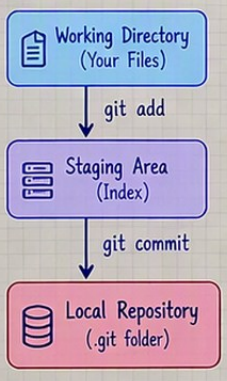
Git is a Distributed Version Control System (DVCS)

3 Why Git ?

- Open source and widely adopted
- Fast and lightweight
- Distributed (full history on local machine)
- Excellent branching and merging
- Works offline
- Large community support
- Used by platforms like GitHub, GitLab, Bitbucket
- Suitable for small to large projects
- Industry standard for DevOps and CI/CD

"Git gives you control, flexibility and collaboration – all in one !"

4 Git Architecture Overview



Three main areas in Git workflow !

- **Working Directory**
  - Where you create/ modify files.
- **Staging Area**
  - Files are added here before commit (using git add).
- **Local Repository**
  - Stores the committed changes in .git folder.

5 Git Installation

- Download Git from official website:
  `https://git-scm.com`
- Install based on your OS:
  - Windows : Download and run installer
  - Linux : sudo apt install git
  - macOS : brew install git
- Verify installation:

```
$ git --version
# Output: git version 2.x.x
```

Once installed, Git can be used from terminal or Git Bash (Windows) !

6 Git Configuration

- Set your username and email address (used in commits).
- These details identify you in the commit history.

Commands:

```
$ git config --global user.name "Your Name"
$ git config --global user.email "youremail@example.com"
```

Check configuration:

```
$ git config --list
```

Use the same email as your GitHub/GitLab account !

7 Practice Commands

1. Check Git version
   ```
   $ git --version
   ```
2. Set your name
   ```
   $ git config --global user.name "Your Name"
   ```
3. Set your email
   ```
   $ git config --global user.email "youremail@example.com"
   ```
4. Verify configuration
   ```
   $ git config --list
   ```

Try these commands in your terminal and verify the output !

8 Common Issues & Solutions

Issue	Possible Cause	Solution
git not found	Git not installed or not in PATH	Install Git and check PATH
Permission denied	Insufficient permissions	Run terminal as admin (or use sudo)
Name/email not set	Not configured	Use git config --global user.name / user.email
Wrong email	Multiple accounts	Update with correct email
Command not working	Older Git version	Upgrade Git

9 Interview Tips

- ☑ What is version control?
- ☑ Difference between centralized and distributed VCS?
- ☑ Why is Git so popular?
- ☑ Explain the Git workflow (working dir → staging → repo).
- ☑ How do you install Git?
- ☑ How do you configure username and email?
- ☑ How do you check the Git version?
- ☑ Where is the .git folder stored?
- ☑ Can you work offline in Git?
- ☑ What happens if you don't configure user.name and user.email?

10 Key Takeaways

- ☑ Version control helps you track changes and collaborate.
- ☑ Git is a distributed VCS and industry standard.
- ☑ Main areas: Working Directory, Staging Area, Local Repository.
- ☑ Install and configure Git before using it.
- ☑ Use correct username and email for commits.
- ☑ Practice basic commands to get comfortable.
- ☑ It's the foundation for advanced Git topics (branching, merging, etc.).

Good Foundations lead to Great Projects !

"Start with the basics, the rest will be easy !"

Version Control Today → Better Collaboration Tomorrow !

Learn | Practice | Build | Grow

Learn
Practice
Build
Be Confident !

Day 2: Git Repository Basics

From Local to Commit – Track Your Changes !

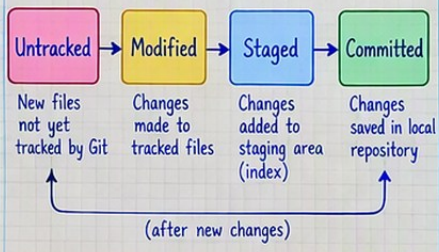
Small Commits Make Big Progress !

1 What is a Git Repository ?

- A repository (repo) is a directory that contains your project files and the complete history of changes.
- It can be local (on your machine) or remote (e.g. GitHub, GitLab).
- It tracks every modification, who made it and when.

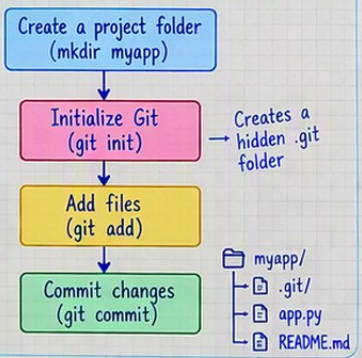
"A repo is like a time machine for your code !"

2 Git File States (Lifecycle)



Use "git status" to check the current state of files.

3 Local Repository Setup Flow



4 Important Git Commands

Command	Purpose
git init	Initialize a new local repository
git clone <url>	Clone a remote repository
git status	Show the status of working directory
git add <file>	Add file(s) to staging area
git commit -m "msg"	Commit staged changes
git log	Show commit history
git diff	Show changes between commits, files or working directory

5 Command Examples

Initialize a new repository:

```
$ mkdir myapp
$ cd myapp
$ git init
```

Clone an existing repository:

```
$ git clone https://github.com/user/repo.git
```

Check status:

```
$ git status
```

Add and commit files:

```
$ git add app.py
$ git commit -m "Initial commit"
```

6 git log and git diff

View commit history:

```
$ git log
$ git log --oneline --graph --all
```

Use --oneline --graph for a cleaner view!

Check differences:

```
$ git diff
  # see changes in working directory
$ git diff --staged
  # see changes in staged files
$ git diff <commit1> <commit2>
  # compare two commits
```

7 Hands-on Practice

Practice these commands on your own laptop !

1. Create a local repository

```
$ mkdir demo-repo
$ cd demo-repo
$ git init
```

2. Add files

```
$ echo "Hello Git" > README.md
$ git add README.md
```

3. Commit changes

```
$ git commit -m "Add README file"
```

4. View commit history

```
$ git log --oneline
```

8 Common Mistakes & Fixes

Mistake	Solution
Forgetting to add files	Run git add <file> before commit.
Empty commit message	Use a meaningful message with -m option.
Committing sensitive files	Use .gitignore to exclude unwanted files.
Confusing working directory and staging	Use git status to check the state.
Accidentally committing to wrong branch	Check current branch with git branch or git status.
Large commits	Make small, logical commits with clear messages.

9 Interview Tips

Be ready with real examples !

- [x] What is a Git repository?
- [x] Difference between git init and git clone?
- [x] What happens when you run git add?
- [x] What does git commit do?
- [x] How to check the status of files?
- [x] How to view commit history?
- [x] What is the difference between git diff and git log?
- [x] Where is the .git folder stored?
- [x] Can we have multiple local repositories on the same machine?
- [x] How do you clone a specific branch?

10 Key Takeaways

- [x] A Git repository tracks all changes to your code.
- [x] Use git init for new projects and git clone to copy existing ones.
- [x] Understand the file states: untracked → modified → staged → committed.
- [x] Use git status, git log and git diff regularly.
- [x] Write clear and meaningful commit messages.
- [x] Practice these commands to build confidence.
- [x] Version control is essential for collaboration and DevOps workflows.

"Good developers write code.
Great developers track their changes !"

git

Learn | Practice | Grow

Learn
Practice
Build
Be Confident !

Day 3: Branching Basics

git

Branch → Experiment → Collaborate → Build Better !

Branches keep your ideas safe !

1 What is a Branch ?

- A branch is a lightweight, movable pointer to a commit.
- It allows you to work on new features, fixes or experiments without affecting the main code (default branch).
- Each branch has its own history of commits.
- Branches make collaboration easier and safer.

"A branch is your safe space to try, break and build !"

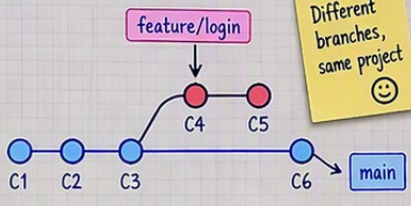
2 Default Branch (main / master)

- Modern repositories use main as the default branch (Instead of master).
- The default branch represents the stable version of the project.
- All new branches are created from the default branch (usually main).

main (recommended)

master (older projects)

3 Visualizing Branches



Different branches, same project

C1, C2, C3 → commits on main
C4, C5 → commits on feature branch
C6 → new commit after merge

4 Common Branch Commands

Command	Purpose
git branch	List all local branches
git branch <name>	Create a new branch
git checkout <name>	Switch to a branch (old way)
git switch <name>	Switch to a branch (new way - recommended)
git branch -d <name>	Delete a branch (safe delete - merged)
git branch -D <name>	Force delete a branch (not merged)

5 Creating a New Branch

```
# Create a branch
$ git branch feature/login

# or create and switch in one step
$ git switch -c feature/login
```

Use clear and meaningful branch names !

Naming Conventions (examples)

- feature/login
- feature/user-profile
- bugfix/fix-login-issue
- hotfix/critical-fix
- release/v1.0

Good names make your team happy !

6 Switching Between Branches

```
# Switch to an existing branch
$ git switch feature/login

# Old way (still works)
$ git checkout feature/login

# Switch back to main
$ git switch main
```

Use git switch in new projects !

When you switch branches, your working directory also changes to the latest code in that branch.

7 Listing Branches

```
# List local branches
$ git branch

# List remote branches
$ git branch -r

# List all branches (local + remote)
$ git branch -a
```

* indicates the current branch !

Example Output:

```
* main          (current branch)
  feature/login
  bugfix/issue-101
  remotes/origin/main
  remotes/origin/feature/login
```

8 Deleting a Branch

```
# Delete a merged branch (safe)
$ git branch -d feature/login

# Force delete (not merged)
$ git branch -D feature/login
```

You cannot delete the current branch. Switch to another branch first !

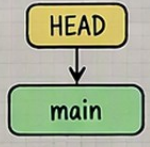
9 Practical Example

```
# 1. Create a new branch
$ git switch -c feature/login

# 2. Make some changes
# (edit files, commit, etc)
$ git add .
$ git commit -m "Add login feature"

# 3. Switch back to main
$ git switch main

# 4. Merge the feature branch
$ git merge feature/login
```

Try this hands-on in your local repo !

10 HEAD Pointer

- HEAD is a pointer to your current branch (or commit).
- When you switch branches, HEAD moves to that branch.
- You can also point HEAD to a specific commit (detached HEAD state).



HEAD tells Git "where you are" in your project !

11 Interview Tips

- [x] What is a branch in Git?
- [x] Why do we use branches?
- [x] What is the default branch in Git?
- [x] Difference between git checkout and git switch?
- [x] How do you create, list and delete branches?
- [x] Can we delete a branch that is not merged?
- [x] What happens if you delete the current branch?
- [x] How does HEAD work?
- [x] Explain a real-world use case for branching (e.g., feature development).

12 Key Takeaways

- [x] Branches are lightweight and powerful.
- [x] Use main as the default branch.
- [x] Create meaningful branch names.
- [x] Use git switch (modern) instead of checkout.
- [x] Delete branches safely after merge.
- [x] Branches enable parallel development.
- [x] Understand HEAD pointer.
- [x] Practice branching and merging regularly.
- [x] Use branches for features, bug fixes and experiments.

Good branching leads to clean history and happy teams !

Branch today, build tomorrow !

Learn | Practice | Collaborate | Grow

Learn
Practice
Build
Be Confident

Day 4 : Merging & Conflicts

git

Combine changes · Resolve conflicts · Keep history clean

Good Merges Lead to Great Code!

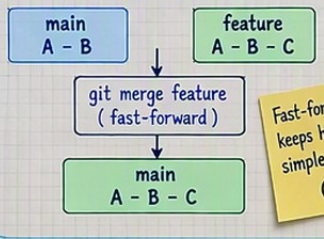
1 What is a Merge ?

- Merging combines the changes from one branch into another.
- It integrates the commit history of the source branch into the target branch.
- Used to bring completed features into main (e.g. merge feature -> main).

"Merge brings teams together and code forward!"

2 Fast-Forward Merge

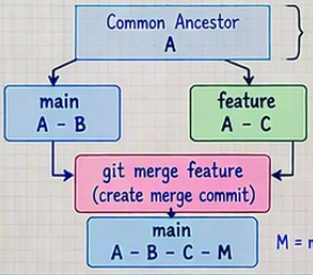
- Occurs when the target branch has no new commits since the source branch was created.
- Git simply moves the branch pointer ahead (no extra merge commit).
- Results in a linear history.



Fast-forward keeps history simple !

3 Three-Way Merge (with merge commit)

- Used when both branches have new commits.
- Git finds a common ancestor and merges the changes from both sides.
- Creates a new merge commit.



M = merge commit

Three-way merge keeps both histories and shows how code evolved !

4 Merge Conflicts

- A conflict occurs when Git cannot automatically merge changes.
- Usually happens when the same lines in a file are modified in both branches.
- Git marks the conflicted sections in the file.

Conflict Markers (in file)

```
<<<<<<< HEAD
  Your changes (e.g. in main)
========
  Changes from feature branch
>>>>>>> feature
```

Resolve conflicts manually by choosing the correct code

5 Resolving Conflicts Manually

1. **Edit the file**
   Open the file and look for conflict markers (<<<<<<<, =======, >>>>>>>).
2. **Resolve the conflict**
   Keep the correct changes or combine both.
3. **Add the file**
   Mark the file as resolved.
   `$ git add <file>`
4. **Commit the changes**
   Complete the merge.
   `$ git commit`

"Read → Edit → Add → Commit → Done!"

6 Useful Commands

Command	Purpose
git merge <branch>	Merge a branch into current branch
git mergetool	Open a visual merge tool to resolve conflicts
git status	Show files with conflicts
git add <file>	Mark file as resolved
git commit	Complete the merge
git log --graph	View merge history

"Use 'git mergetool' for a better visual conflict resolution experience !

7 Abort / Safety

- If you run into issues during a merge, you can abort and go back to the previous state.

```
$ git merge --abort
```

- This will stop the merge process and restore your branch to the state before the merge.
- No changes will be committed.

"When in doubt, abort and try again. Better safe than sorry !

8 Practice Exercises

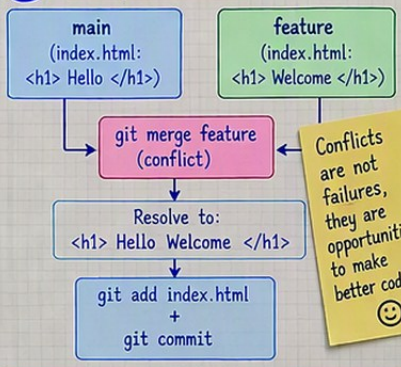
1. **Create a conflict scenario**

```
$ git checkout -b feature
# make changes to a file
$ git commit -m "Update file"
$ git checkout main
# make different changes to same file
$ git commit -m "Update file in main"
$ git merge feature  # should cause conflict
```

2. **Resolve the conflict**
   - Open the conflicted file.
   - Edit and keep the correct changes.
   - $ git add <file>
3. **Commit the resolved changes**

```
$ git commit -m "Resolve merge conflict"
```

9 Example Scenario



Conflicts are not failures, they are opportunities to make better code!

10 Troubleshooting Tips

Issue	Possible Cause	Solution
Unexpected conflicts	Both branches changed same lines	Manually resolve and commit
Merge not allowed	Uncommitted changes	Commit or stash changes first
Accidentally merged wrong branch	Wrong branch name	Use git reflog and reset if needed
Want to cancel merge	Merge in progress	Use git merge --abort
Too many conflicts	Large changes / old branch	Consider rebasing or merging in smaller chunks

Check 'git status" whenever something looks odd !

11 Interview Questions

- [x] What is a merge in Git?
- [x] What is a fast-forward merge?
- [x] What is a three-way merge?
- [x] Why do merge conflicts happen?
- [x] How do you resolve a merge conflict?
- [x] What is the purpose of git mergetool?
- [x] How do you abort a merge?
- [x] How can you identify files in conflict?
- [x] What happens after a successful merge?
- [x] Merge vs Rebase – when to use which?
- [x] How do you keep a clean commit history?
- [x] Real-world scenario discussion.

12 Key Takeaways

- [x] Merge combines changes from branches.
- [x] Fast-forward merge keeps linear history.
- [x] Three-way merge creates a merge commit.
- [x] Conflicts happen when same lines are changed in both branches.
- [x] Resolve conflicts by editing, adding and committing.
- [x] Use git mergetool for easier resolution.
- [x] Use git merge --abort if needed.
- [x] Always test after merge.
- [x] Keep communication within the team.
- [x] Understand merge vs rebase.

"Good merges resolve differences, Great developers build together !"

Learn | Practice | Merge | Grow

Learn
Practice
Build
Be Confident !

Day 5: Remote Repositories

Connect · Collaborate · Push · Pull · Stay in Sync !

Code
Backup
Collaborate
from
Anywhere !

1 What is Remote Repository ?

- A remote repository is a version of your project hosted on a server (e.g. GitHub, GitLab, Bitbucket).
- It allows multiple developers to collaborate, share changes and keep code safe.
- Common name for the main remote is 'origin' (by convention).

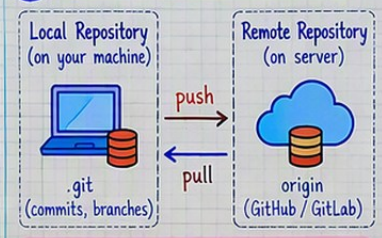
"Remote repo = Central place to share and sync code."

2 Popular Git Hosting Services

GitHub	GitLab	Bitbucket
• Widely used	• Self-hosted option	• Popular in enterprises
• Great for open source	• Good for DevOps	• Integrates with Jira
• Free for public repos	• CI/CD integrated	• Good for private repos

All work the same way with Git commands !

3 Local vs Remote Repository



Keep your local and remote repo in sync !

4 Useful git remote Commands

Command	Purpose
git remote -v	List remote repositories (with URL)
git remote add origin <url>	Add a remote repository
git remote remove origin	Remove a remote
git remote rename old new	Rename a remote
git remote show origin	Show details about a remote

Usually the main remote is named 'origin" but you can use any name !

5 Push, Pull and Fetch (Difference)

Command	What it does ?	Effect on local code
git push	Uploads your local commits to remote	No change
git pull	Fetches + Merges changes from remote	Updates local code
git fetch	Downloads changes from remote (no merge)	No change (only updates remote tracking branches)

Pull = Fetch + Merge

6 Basic Examples

1. Check remote URL

```
$ git remote -v
origin  https://github.com/user/repo.git (fetch)
origin  https://github.com/user/repo.git (push)
```

2. Push local branch to remote

```
$ git push origin main
```

3. Pull latest changes

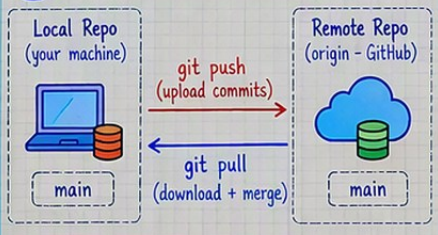
```
$ git pull origin main
```

4. Fetch without merge

```
$ git fetch origin
```

Always check before you push !

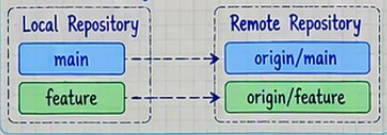
7 How Push and Pull Work ?



Push your changes, Pull others' changes, Keep both in sync !

8 Remote Tracking Branches

- When you fetch or pull, Git creates remote tracking branches (e.g. origin/main).
- These track the state of branches on the remote repository.
- They are read-only and cannot be modified directly.
- Example: origin/main tracks the main branch on origin.



9 Upstream Branches

- An upstream branch is the remote branch that your local branch tracks.
- Allows you to use simple commands like `git pull' and `git push' without specifying the branch every time.
- Set upstream when pushing for the first time:

```
$ git push -u origin feature
```

Set it once, use simple commands after that !

10 Practice Scenarios

1. Push your local repo to GitHub

```
$ git init
$ git remote add origin <repo-url>
$ git push -u origin main
```

2. Pull latest changes from remote

```
$ git pull origin main
```

3. Fetch changes without merge

```
$ git fetch origin
$ git log origin/main
```

4. Work with a new branch

```
$ git checkout -b feature
$ git push -u origin feature
```

Practice these commands in a real repo. It builds confidence !

11 Common Errors & Solutions

Error	Reason	Solution
fatal: not a git repository	Not in a Git repo	Run `git init' or `git clone'
Permission denied (publickey)	SSH key not configured	Add SSH key or use HTTPS
rejected (non-fast-forward)	Remote has new commits	Pull changes first (`git pull')
fatal: 'origin' does not appear to be a git repository	Wrong remote URL	Check URL with `git remote -v'
Everything up-to-date	No new changes	Make new commits and push

12 Interview Tips

- ✅ What is a remote repository?
- ✅ Why is the default remote name `origin'?
- ✅ Difference between push, pull and fetch?
- ✅ What are remote tracking branches?
- ✅ How do you set an upstream branch?
- ✅ How do you handle a non-fast-forward error?
- ✅ What happens when you run `git fetch'?
- ✅ How do you list all remotes?
- ✅ How do you change a remotes URL?
- ✅ Real-world scenario: team collaboration using GitHub?

Git connects people, processes and possibilities !

"Push your work, Pull new ideas, Build something amazing !"

Learn | Practice | Collaborate | Grow

Learn
Practice
Build
Grow !

Day 6: GitHub / GitLab Workflow

GitHub GitLab

Collaborate • Review • Automate • Deliver Better Code !

Good Collaboration Builds Great Software !

1 Fork vs Clone

Fork (GitHub / GitLab)

- Creates a copy of the repository under your account.
- Used when you don't have write access to the original repo (e.g. open source).
- You can propose changes via Pull Request / Merge Request.

Clone

- Creates a local copy of an existing repository.
- Used when you have access to the repository.
- Allows you to work on the code locally.

Original Repo (owner) → Fork → Your Fork (your account)

Clone to local machine

```
git clone <repo-url>
```

Fork = copy on GitHub/GitLab Clone = copy on your machine

2 Pull Request / Merge Request Lifecycle

GitHub (Pull Request)	GitLab (Merge Request)
Create feature branch git checkout -b feature-xyz	Create feature branch git checkout -b feature-xyz
Push branch to GitHub git push origin feature-xyz	Push branch to GitLab git push origin feature-xyz
Open Pull Request (compare branches)	Open Merge Request (compare branches)
Code Review Team reviews the changes	Code Review Team reviews the changes
Approvals & Checks CI/CD checks, required approvals	Approvals & Checks CI/CD checks, required approvals
Merge PR Changes merged to main	Merge MR Changes merged to main

Same concept, different names → PR (GitHub) = MR (GitLab)

3 Branch Protection Rules

- Protects important branches (e.g. main, master) from unwanted changes.
- Prevents direct pushes and enforces code review.
- Ensures only approved and tested code is merged.

Keep your main branch SAFE !

Common Rules

- ✅ Require pull request / merge request before merging
- ✅ Require code review approvals
- ✅ Require status checks to pass (e.g. CI/CD)
- ✅ Restrict who can push to the branch
- ✅ Require signed commits (optional)
- ✅ Prevent force pushes
- ✅ Require linear history (optional)

4 Code Reviews

- Code review is the process of reviewing changes before merging.
- Helps find bugs, improve code quality and share knowledge.
- Both GitHub and GitLab provide in-built tools for discussions, comments and suggestions.

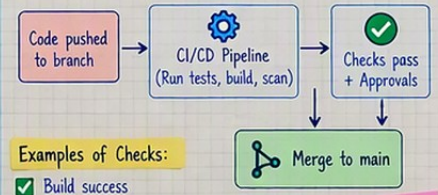
Tips for Good Reviews

- ✅ Review logic, not just syntax
- ✅ Provide constructive feedback
- ✅ Suggest improvements
- ✅ Check for security issues
- ✅ Keep discussions respectful

Review today, better code tomorrow !

5 Approvals and Checks

- You can require a minimum number of approvals before merging.
- Integrate with CI/CD (GitHub Actions / GitLab CI) to run automated tests.
- Merging is allowed only when all checks pass.



Examples of Checks:

- ✅ Build success
- ✅ Unit / Integration tests
- ✅ Code quality (linting, SAST)
- ✅ Security scans (e.g. dependency check)

No green checks, No merge !

6 Useful Commands

Use CLI tools to be more productive !

GitHub (PR)

Command	Purpose
git push origin <branch>	Push your branch
Open PR (UI)	Create Pull Request
gh pr create	Create PR using Gitlab CLI
gh pr list	List PRs
gh pr review	Review a PR
gh pr merge	Merge a PR

GitLab (MR)

Command	Purpose
git push origin <branch>	Push your branch
Open PR (UI)	Create Merge Request
glab mr create	Create MR using GitLab CLI
glab mr list	List MRs
glab mr review	Review a MR
glab mr merge	Merge a MR

7 Practice: End-to-End Workflow

1. Fork a repository (or use your own repo).
2. Clone the repository to local machine.

```
git clone <repo-url>
cd <repo>
```

3. Create a new branch.

```
git checkout -b feature-xyz
```

4. Make changes and commit.

```
git add .
git commit -m "Add new feature"
```

5. Push the branch.

```
git push origin feature-xyz
```

6. Open a Pull Request / Merge Request on GitHub or GitLab.
7. Ask for review, address feedback.
8. Merge the PR/MR.

Practice more → Become confident !

8 Common Scenarios & Solutions

Scenario	Solution
Need to contribute to open source project	Fork the repo, create a branch, raise a PR
PR shows conflicts	Pull latest changes, resolve conflicts locally, push again
Checks are failing.	Check CI/CD logs, fix the issue, push new commits
Need to update PR	Make changes in the same branch and push again
Want to cancel a PR/MR	Close the PR/MR (UI or CLI)
Accidentally pushed to main	Revert the commit or use branch protection to avoid direct pushes
Need to request specific reviewers	Use 'Reviewers' section in PR/MR
Want to squash commits	Squash and merge (UI) or rebase before merge

Real problems teach the best lessons !

9 Interview Tips

- ✅ What is the difference between fork and clone?
- ✅ Explain the Pull Request / Merge Request flow.
- ✅ Why are code reviews important?
- ✅ What are branch protection rules?
- ✅ How do you enforce approvals and checks?
- ✅ How do you resolve conflicts in a PR/MR?
- ✅ What CI/CD tools integrate with GitHub/GitLab?
- ✅ How do you restrict direct pushes to main?
- ✅ What happens if a build fails in a PR?
- ✅ How do you squash commits before merging?
- ✅ Explain GitHub Actions / GitLab CI in brief.
- ✅ Real-world scenario: handling a large feature branch.

10 Key Takeaways

- ✅ Use fork for open source contributions.
- ✅ Follow branching strategy (feature branches).
- ✅ Always raise a PR/MR for code changes.
- ✅ Enforce code reviews and CI/CD checks.
- ✅ Protect your main branch.
- ✅ Use CLI tools (gh / glab) for efficiency.
- ✅ Practice real-world workflows.
- ✅ Good collaboration leads to better software !

Collaborate Learn Improve Repeat !

"Better Habits, Better Code, Brighter Opportunities !"

Learn | Practice | Collaborate | Grow

Learn
Practice
Build
Get Confident !

Day 7: Rebase - Very Important

Rewrite history to keep a clean, linear commit history !

git

Clean History Happy Team

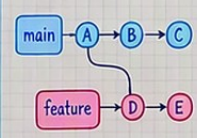
1 What is Rebase ?

- Rebase moves your commits to a new base commit.
- It replays your commits on top of another branch (usually the latest main).
- It creates a linear history (no merge commit).
- Useful to keep history clean before merging.
- Does not change the content of your changes, only the commit history.

"Rebase = Take my commits, reapply them on top of latest code !"

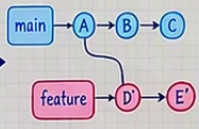
2 Rebase - Before and After

Before Rebase (feature behind main)



Feature branch is behind main and has its own commits.

After Rebase (feature on latest main)



Feature commits are replayed on top of latest main. (clean linear history)

"Same changes, different commit IDs !"

3 Merge vs Rebase

Feature	Merge	Rebase
History	Creates a merge commit (non-linear)	Linear clean history
Commit IDs	Keeps original commit IDs	Creates new commit IDs
Use case	Preserve history, collaboration	Clean history (before merge)
Complexity	Simple	Needs care (conflicts possible)
Recommended	For shared/public branches	For local feature branches
Example	git merge feature	git rebase main

4 Interactive Rebase (git rebase -i)

- Allows you to rewrite, edit, reorder or squash commits.
- Opens an editor with a list of recent commits.
- You can change commands like pick, squash, reword, edit, drop.

Example:

```
git rebase -i HEAD~4
                # last 4 commits
```

Clean history before creating a Pull Request !

Commands in editor:

```
pick    = keep commit
reword  = change commit message
edit    = stop and edit commit
squash  = combine with previous commit
fixup   = combine with previous (discard message)
drop    = remove commit
```

5 Squashing Commits

- Squash combines multiple commits into one commit.
- Useful to keep a clean and meaningful commit history.
- Commonly done using interactive rebase.

Example: Squash last 3 commits

1. git rebase -i HEAD~3
2. Change second and third commit from pick to squash
3. Save and update the commit message
4. Result: 3 commits → 1 commit

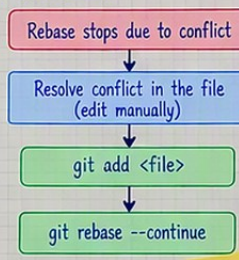
Before:

After (squash B,C,D):

Write clean, meaningful commits !

6 Rebase Conflicts and Resolution

- During rebase, if the same file is changed in both branches, conflicts can occur.
- Git will pause and ask you to resolve conflicts for each commit.



To cancel rebase:

```
git rebase --abort
```

Conflicts are normal. Read the error, fix and continue!

7 Common Rebase Commands

Command	Purpose
git rebase <branch>	Rebase current branch onto <branch>
git rebase -i <commit>	Interactive rebase
git rebase --continue	Continue after resolving conflicts
git rebase --abort	Cancel rebase and go back
git rebase --skip	Skip current commit (in conflict)
git rebase --onto <new> <old>	Advanced rebase (move commits)

8 Practical Examples

1. Clean commit history before PR

```
git checkout feature
git rebase main
git rebase -i HEAD~3  # squash if needed
git push -f    # if not pushed yet
```

2. Squash commits

```
git rebase -i HEAD~5
# change pick to squash
```

3. Rebase feature branch

```
git checkout feature
git rebase main
# resolve conflicts if any
git push (or push -f if local branch)
```

Practice to get comfortable !

9 Important Warning

DO NOT rebase shared or public branches that others are using (e.g. main, develop).

- Rebasing changes commit IDs.
- It can cause confusion and break others' work.
- Only rebase your local feature branches (before pushing or before creating PR).
- If the branch is already pushed and shared, prefer merge instead of rebase.

"Never rewrite public history unless you are 100% sure !"

10 Interview Questions

Be ready with examples !

- [x] What is rebase and why is it used?
- [x] Difference between merge and rebase?
- [x] When should you use rebase?
- [x] How do you squash multiple commits?
- [x] How do you resolve conflicts during rebase?
- [x] What happens to commit IDs after rebase?
- [x] Can we rebase a branch that is already pushed?
- [x] What is interactive rebase and what can you do with it?
- [x] How do you cancel a rebase?
- [x] Real-world scenario: clean commit history before PR.

11 Key Takeaways

- [x] Rebase replays commits on top of another branch.
- [x] Creates a clean, linear history.
- [x] Use interactive rebase to edit, squash or reorder.
- [x] Resolve conflicts carefully and continue.
- [x] Do not rebase shared/public branches.
- [x] Use rebase for local feature branches.
- [x] Practice squash to keep meaningful commits.
- [x] Understand the difference between merge and rebase.
- [x] Rebase is powerful but use it responsibly.

"Clean history today, Easier maintenance tomorrow !"

Keep Learning
Keep Practicing
Master Git !

Learn
Practice
Fix Mistakes
Be Confident !!

Day 8: Undoing & Fixing Mistakes

git

Mistakes happen ... Git gives you a second chance !

Stay Calm
Understand
Choose the right tool
Fix it !

1 Why do mistakes happen ?

- Wrong commit message
- Committed the wrong files
- Need to undo local changes
- Accidentally deleted a branch/commit
- Pushed something you didn't intend to
- Need to go back to a previous state

Good news: Git keeps history ! You can almost always recover.

2 git reset (soft, mixed, hard)

- Moves the current branch (HEAD) to a previous commit. Changes how HEAD, index and working tree are updated.

Mode	HEAD	Index (Staging)	Working Directory	Use Case
--soft	Moves	No change	No change	Undo last commit (keep changes staged)
--mixed (default)	Moves	Reset	No change	Undo last commit (keep changes in working directory)
--hard	Moves	Reset	Reset	Discard changes completely

3 git revert

- Creates a new commit that undoes the changes from a previous commit.
- Does not change history.
- Safe to use on shared branches.

Example:

```
git revert <commit-hash>
# or
git revert HEAD
```

Use git revert for commits that are already pushed to remote.

4 git checkout (file)

- Restore a file from a specific commit (to working directory).
- Useful in older Git versions.
- Can also be used to switch branches (in older versions).

Bring back a single file without affecting others !

Example:

```
git checkout <commit-hash> -- <file>
# Restore file from last commit
git checkout -- <file>
```

5 git restore (newer, recommended)

- Modern way to restore files.
- Clearer and more intuitive than checkout.
- Can restore from working directory, index, or a specific commit.

Examples:

```
# Discard changes in working directory
git restore <file>
# Restore file from a specific commit
git restore --source <commit> -- <file>
# Unstage a file
git restore --staged <file>
```

6 git reflog (Your Safety Net)

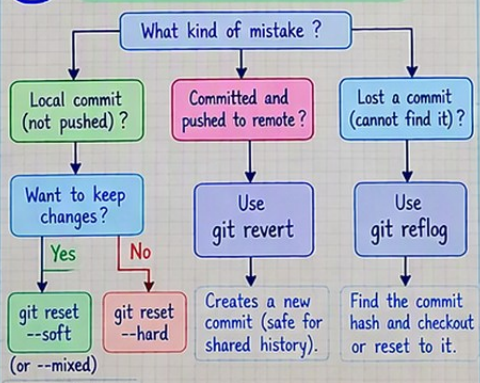
- Keeps a record of where HEAD has been.
- Helps recover lost commits, even if they are deleted.
- Each action (commit, reset, merge, etc.) is recorded.

Examples:

```
git reflog      # View history of HEAD
git checkout <commit-hash>
```

If you lose a commit, don't panic ! Use git reflog to find it and restore.

7 Decision Tree - What to use ?



Not sure ? Use reflog to explore !

8 Practical Examples

1. Undo last commit (keep changes)
   `git reset --soft HEAD~1`
2. Undo last commit (keep changes in working dir)
   `git reset --mixed HEAD~1`
3. Discard last commit (lose changes)
   `git reset --hard HEAD~1`
4. Revert a committed change
   `git revert <commit-hash>`
5. Restore a specific file
   `git restore --source HEAD~2 app.py`
6. Unstage a file
   `git restore --staged app.py`
7. Recover a deleted commit
   `git reflog`
   `git checkout <commit-hash>`

9 Common Scenarios & Solutions

Scenario	Solution
Undo last commit (but keep changes)	git reset --soft HEAD~1
Undo last commit (keep in working dir)	git reset --mixed HEAD~1
Discard all local changes	git reset --hard HEAD
Undo a pushed commit	git revert <commit-hash>
Restore a deleted file	git restore <file>
Unstage a file	git restore --staged <file>
Find lost commit	git reflog
Checkout previous version of a file	git checkout <commit> -- <file>

Always double-check the commit hash before running reset or revert !

10 ⚠ Important Warnings

- git reset --hard permanently discards changes in working directory. Use with caution !
- Avoid using --hard on important work.
- Prefer git revert for shared branches.
- Always verify commit hash before resetting.
- Use git reflog if you think you lost something.
- Don't force push after rewriting history unless you are sure (use with team communication).

Be careful with --hard ... It's powerful but dangerous !

11 Interview Tips

- [x] Explain difference: reset vs revert.
- [x] When would you use --soft, --mixed, --hard?
- [x] How does reflog help in recovery?
- [x] What is the difference between checkout and restore?
- [x] How do you undo a pushed commit?
- [x] Is it safe to use reset on a shared branch?
- [x] How would you recover a deleted branch/commit?
- [x] What happens to working tree, index and HEAD in different reset modes?
- [x] Real-world scenario: accidentally committed secrets - how do you fix it?

Explain the reasoning, not just the command !

12 Key Takeaways

- [x] Git gives you multiple ways to undo and fix mistakes.
- [x] Choose the right tool based on the situation.
- [x] Use reset for local, revert for shared history.
- [x] Use restore/checkout for files.
- [x] Reflog is your safety net.
- [x] Always be careful with --hard.
- [x] Practice these scenarios hands-on !

Mistakes are part of the learning journey. What matters is how you fix them !

Keep Learning | Keep Practicing | You've Got This !

Learn
Practice
Build
Grow !

Day 9: Tags & Releases

git

Mark important versions · Manage releases · Deploy with confidence !

Tags are snapshots of stability ! Not just another branch !

1 What are Tags ?

- A tag is a reference to a specific commit in Git.
- Used to mark important points such as releases (v1.0, v1.1, v2.0, etc.).
- Tags do not change (unlike branches).
- Commonly used for versioning and deployments.

Tags are like a bookmark for a commit !

2 Types of Tags

Lightweight Tag	Annotated Tag
• Simple reference to a commit. • No extra information. • Created quickly. • Usually used for temporary or internal tagging.	• Stores additional metadata (name, email, date, message). • More informative. • Recommended for releases. • Can be signed (GPG).
Command: `git tag v1.0`	Command: `git tag -a v1.0 -m "Initial release"`

Use annotated tags for official releases !

3 Versioning Example

Follow semantic-style versioning:

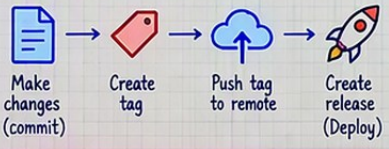
- v1.0 → First stable release
- v1.1 → Minor update (new features)
- v1.2.0 → Patch update (bug fixes)
- v2.0 → Major release (breaking changes)

Example commit history:

```
a1b2c3  (Initial commit)
b2c3d4  (Add login feature)
c3d4e5  (Fix bug)
d4e5f6  (Add new feature)
e5f6g7  (Release v1.0)  ← tag: v1.0
f6g7h8  (More changes)
g7h8i9  (Release v1.1)  ← tag: v1.1
```

Each tag points to a specific commit !

4 Release Workflow



Commit → Tag → Push → Release → Deploy

Tags help automate and track releases in CI/CD pipelines !

5 Common Commands

List all tags	`git tag`
Create lightweight tag	`git tag v1.0`
Create annotated tag	`git tag -a v1.0 -m "Initial release"`
Show tag details	`git show v1.0`
Push a single tag	`git push origin v1.0`
Push all tags	`git push --tags`
Delete a local tag	`git tag -d v1.0`
Delete a remote tag	`git push origin --delete v1.0`

6 Practice: Create and Push a Tag

1. Make some commits
2. Create an annotated tag
   `git tag -a v1.0 -m "Release v1.0"`
3. Verify the tag
   ```
   git tag
   git show v1.0
   ```
4. Push the tag to remote
   ```
   git push origin v1.0
   # or push all tags
   git push --tags
   ```

Try this in your own repository !

Check the tag on GitHub in Releases tab !

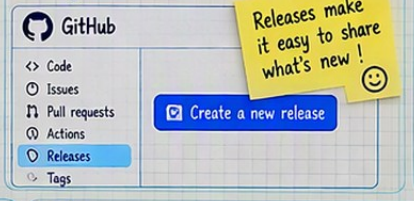
7 Tags vs Branches

Feature	Tag	Branch
Purpose	Mark a specific release	Ongoing development
Moves?	No (fixed to a commit)	Yes (moves with new commits)
Common use	Releases, versioning	Features, bug fixes
Can have multiple?	Yes	Yes
Recommended for	Stable versions	Active development

Use branches to develop, tags to release !

8 Releasing on GitHub / GitLab

- Push your tag to remote repository.
- Create a release (on GitHub/GitLab).
- Add release notes (what's new, bug fixes, etc.).
- Attach artifacts (if any).
- This helps the team and users track versions easily.



Releases make it easy to share what's new !

9 Real-World Use Cases

- ✅ Marking production releases (v1.0, v2.0).
- ✅ Triggering deployment pipelines using tags.
- ✅ Rolling back to a specific version.
- ✅ Tracking versions of libraries and tools.
- ✅ Using tags for artifact versioning (Docker images).
- ✅ Automating changelog generation.

Tags = Stability = Reliable Deployments !

10 Troubleshooting

Issue	Possible Cause	Solution
Tag not showing in remote	Not pushed	`git push origin <tag>` or `git push --tags`
Deleted tag still visible	Cache / Not deleted from remote	`git push origin --delete <tag>`
Wrong commit tagged	Tagged wrong commit	Delete and recreate tag with correct commit
Cannot create tag	Tag already exists	Use a different tag name or delete old one

Always verify tags before pushing to remote !

11 Interview Questions

- ✅ What is a tag in Git?
- ✅ What is the difference between lightweight and annotated tags?
- ✅ How do you create and push a tag?
- ✅ When would you use tags?
- ✅ How do you delete a tag (local and remote)?
- ✅ How do tags help in CI/CD?
- ✅ Explain semantic versioning.
- ✅ What is the difference between a tag and a branch?
- ✅ How do you view tag details?
- ✅ Can a tag be moved? Why or why not?

Tags show your commitment to quality !

12 Key Takeaways

- ✅ Tags mark specific commits.
- ✅ Use annotated tags for releases.
- ✅ Tags do not move (unlike branches).
- ✅ Follow proper versioning (v1.0, v1.1, etc.).
- ✅ Push tags to share with the team.
- ✅ Use tags in CI/CD for automated deployments.
- ✅ Release notes improve collaboration.
- ✅ Verify tags before pushing.
- ✅ Delete tags carefully if needed.
- ✅ Tags help maintain a clean and professional release history.

Well tagged code is well maintained code !

Code → Commit → Tag → Release → Deploy → Success !

Learn | Practice | Improve | Grow

Learn
Practice
Automate
Grow !

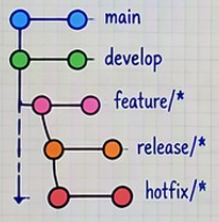
Day 10: Git for DevOps Pipelines

Code → Commit → Build → Test → Deploy → Repeat !

Good Code
+
Good Pipeline
=
Faster Delivery !

1 Git Branching Strategies

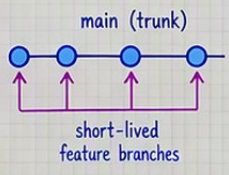
GitFlow (feature-driven)



- Uses multiple branches.
- Stable main branch.
- develop for integration.
- Feature, release and hotfix branches.
- Good for large teams and complex projects.

More control but more complex !

Trunk-Based Development



- Developers work on short-lived branches.
- Frequent commits and merges to main.
- Simplifies workflow.
- Good for CI/CD and modern DevOps teams.

Simpler, faster and works well with CI/CD !

2 Commit Message Standards

- Write clear, concise and meaningful commit messages.
- Common format (Conventional Commits):

<type>(<scope>): <short description>

Common Types:

feat	– New feature
fix	– Bug fix
docs	– Documentation
style	– Code style changes
refactor	– Code refactoring
test	– Adding tests
chore	– Maintenance tasks

Example:

```
feat(api): add user login endpoint
fix(ci): update pipeline config
docs: update README
chore: bump version to v1.2.0
```

"Good commits tell a story !"

3 Repository Structure for DevOps

```
my-app/
├── .github/      # CI/CD workflows
├── src/          # Application code
├── k8s/          # Kubernetes manifests
├── docker/       # Dockerfile, compose
├── helm/         # Helm charts
├── scripts/      # Build & deployment scripts
├── infra/        # Terraform / IaC
├── docs/         # Documentation
├── .gitignore
├── README.md
└── LICENSE
```

Keep your repo organized for scalability !

4 Git Triggers in CI/CD

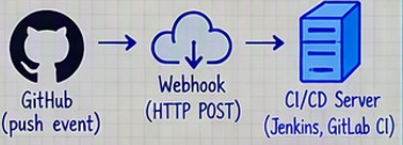
- CI/CD tools (Jenkins, GitLab CI, GitHub Actions, Azure DevOps) can be triggered by Git events.
- Common triggers:

Event	Use Case
Push to branch	Run build & tests
Pull/Merge request	Run validation checks
Tag push	Create release / deployment
Branch creation	Trigger environment setup
Branch deletion	Cleanup resources

Automate your pipeline based on Git events !

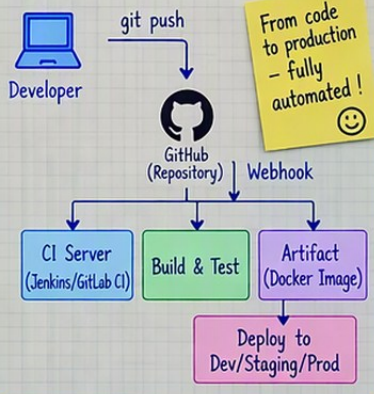
5 Webhooks

- Webhooks notify external services when an event occurs in Git (e.g. push, PR, tag).
- Used by CI/CD tools to trigger pipelines automatically.
- Configure webhook URL in GitHub/GitLab.



Webhooks = Real-time notification for Git events !

6 End-to-End CI/CD Flow



From code to production – fully automated !

7 Practical Scenarios

1 Branch-based pipeline
- Trigger build on push to main or develop.
- Deploy to dev environment automatically.

2 Tag-based deployment
- When a tag (e.g. v1.0) is pushed, trigger release pipeline.
- Deploy to staging/production.

3 Environment-based repositories
- Separate repos for dev, staging, prod (or use branches).
- Use different pipeline configs per environment.

Practice these scenarios in Jenkins / GitLab CI / GitHub Actions !

8 Tradeoffs: GitFlow vs Trunk-Based

Aspect	GitFlow	Trunk-Based
Complexity	Higher	Lower
Number of branches	Multiple	Single (main)
Release cycle	Scheduled	Continuous
Merge conflicts	Higher	Lower (small changes)
Ease of CI/CD	Moderate	Easier
Best for	Large teams, complex products	Modern DevOps, microservices
Learning curve	Steeper	Simpler

Choose the strategy that fits your team and project needs !

9 Interview Tips

- ☑ Explain GitFlow and Trunk-based workflows.
- ☑ What are Git triggers in CI/CD?
- ☑ How do webhooks work?
- ☑ How do you handle deployments using tags?
- ☑ What should be included in a commit message?
- ☑ How do you structure a repo for DevOps?
- ☑ How do you secure Git repos?
- ☑ What is the difference between a branch and a tag in pipelines?
- ☑ Real-world scenario: set up a pipeline for a microservices app.

Show hands-on examples in interviews !

Better Code → Faster Delivery → Happier Teams !

Learn | Practice | Automate | Grow

Learn
Practice
Solve
Build
Grow !

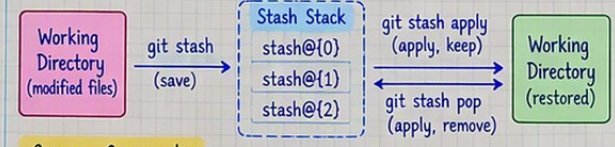
Day 11: Advanced Git - Interview Level

Master Advanced Git Concepts for Real-World Scenarios !

Same Commands Different Possibilities !

1 Git Stash (Save your work temporarily)

- Used to temporarily save uncommitted changes and clean your working directory.
- Useful when you need to switch branches quickly without committing.
- Stash saves changes in a stack (LIFO).



Common Commands:

```
git stash          # save changes
git stash list     # list all stashes
git stash apply    # apply latest stash (keep it)
git stash pop      # apply and remove latest stash
git stash drop     # remove a specific stash
git stash clear    # remove all stashes
```

Stash is like a clipboard for your changes !

2 Cherry-pick (Pick a specific commit)

- Used to apply a specific commit from one branch to another branch.
- Useful when you want a particular fix or feature without merging the entire branch.
- Creates a new commit with the same changes (different commit ID).

Command:

```
git cherry-pick <commit-hash>
```

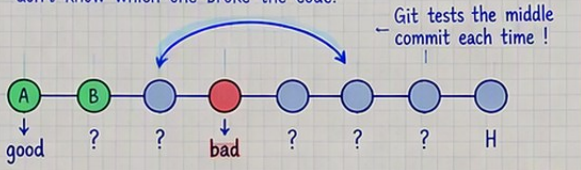
Example:

```
git cherry-pick a1b2c3d  # apply commit a1b2c3d
```

Cherry-pick brings only the change you need !

3 Git Bisect (Find the bug using binary search)

- Helps to find the commit that introduced a bug.
- Works like binary search - you mark commits as good or bad, and Git finds the exact commit.
- Very useful when you have many commits and don't know which one broke the code.



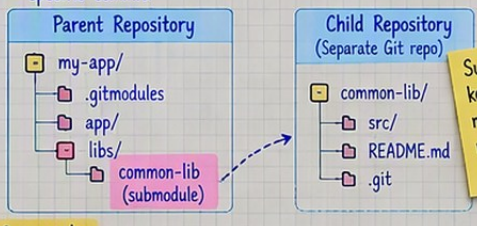
Commands:

```
git bisect start
git bisect bad          # mark current commit as bad
git bisect good <commit-hash>  # mark a good commit
# Git checks out a commit for you to test
git bisect reset        # exit bisect mode
```

Let Git do the detective work !

4 Git Submodules (Use another Git repo inside your repo)

- Allows you to include and manage a Git repository inside another repository.
- Useful for reusable components, libraries or shared code.
- The submodule has its own history and is referenced by a specific commit.



Submodules keep your code modular and reusable !

Commands:

```
git submodule add <repo-url> <path>      # add a submodule
git submodule update --init --recursive  # clone submodules
git submodule status                     # check status
git submodule foreach git pull           # update all submodules
```

5 Mono-repo vs Multi-repo (Which one to choose?)

Aspect	Mono-repo (Single repo)	Multi-repo (Multiple repos)
Definition	All projects in one repository	Each project in a separate repository
Management	Easier to manage at scale	Independent management
CI/CD	Single pipeline (can be complex)	Separate pipelines per project
Dependencies	Shared easily	Needs versioning / releases
Access Control	Common for all projects	Fine-grained access
Best for	Large organizations, microservices, shared code	Small teams, independent services
Examples	Google, Microsoft	Many open-source projects

When to choose?

- Mono-repo → When code is tightly coupled.
- Multi-repo → When projects are independent.

There is no one-size-fits-all. Choose based on your use case !

6 Practice: Hands-on Examples

1. Apply a stash

```
echo "some change" >> file.txt
git stash
git stash list
git stash pop
```

2. Cherry-pick a commit

```
git log --oneline    # get commit hash
git cherry-pick <commit-hash>
```

3. Find a bug using bisect

```
git bisect start
git bisect bad
git bisect good <good-commit-hash>
# Test the checked out commit
git bisect reset
```

Practice these commands in your local repo. Real experience builds confidence !

7 Interview Gold (Be Ready!)

- [x] What is git stash and when do you use it?
- [x] How is cherry-pick different from merge?
- [x] How does git bisect work?
- [x] What are submodules and when would you use them?
- [x] Mono-repo vs multi-repo - pros and cons?
- [x] How do you handle conflicts during cherry-pick or rebase?
- [x] Can we stash untracked files?
- [x] How do you update submodules to the latest version?
- [x] Give a real-world scenario where you used these features.

They don't just ask commands, they ask SCENARIOS !

"Advanced Git skills make you a better developer and a stronger DevOps engineer !"

Commit Knowledge Today → Build Better Tomorrow !

Learn
Practice
Secure
Build
Grow !

Day 12: Git Security & Best Practices

Keep your code safe * Prevent leaks * Follow best practices * Build trust

Secure
Code
Better
Teams

1 Secrets in Git (What NOT to commit?)

- Never commit sensitive information such as:
  - Passwords, API keys, tokens
  - Cloud credentials (AWS, Azure, GCP)
  - Private certificates / SSH keys
  - Database connection strings
  - .env files, config files with secrets
  - Personal data or internal info

Even if you delete a secret from your files, it still exists in the Git history ! Always rotate the leaked secret (generate a new one).

2 .gitignore (Exclude unwanted files)

- **.gitignore** tells Git which files/folders to ignore (not track).
- Helps avoid committing secrets, build files, dependencies, OS files, etc.
- Each line is a pattern (glob).

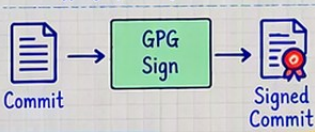
Example .gitignore

```
# Environment files
.env
*.env
# IDE / OS files
.vscode/
.idea/
*.swp
# Dependencies / Build artifacts
node_modules/
target/
dist/
# Logs
*.log
```

Keep your repo clean and focused !

3 GPG Signing (Verify Commits)

- GPG (GNU Privacy Guard) is used to sign commits and tags.
- Helps averify that the commit is from a trusted author and not tampered.
- Adds cryptographic signature to commits.



Common Commands:

```
# Generate a GPG key
gpg --full-generate-key
# Configure Git to use your key
git config --global user.signingkey <KEY_ID>
git config --global commit.gpgsign true
```

Sign your work Show it's you !

4 Access Control (Who can do what?)

- Use proper access control in Git platforms (GitHub, GitLab, Bitbucket).
- Follow the principle of least privilege.
- Use roles like:

Role	Permissions
Read	Clone / Pull (see code)
Write	Push to allowed branches
Maintain	Manage repo, PRs, settings
Admin	Full control (use carefully)

- Use SSO / MFA for better security.
- Regularly review and remove unused access.

Give minimum access needed !

5 Branch Protection (Keep main safe)

- Protect important branches (e.g. main) from direct pushes.
- Require Pull Requests (PR) with review.
- Enforce status checks (CI/CD).
- Optional: require signed commits, code owners review, conversation resolution.

Typical Branch Protection Rules

- [x] Block direct pushes to main
- [x] Require PR reviews (e.g. 1 or 2 approvals)
- [x] Require successful CI/CD checks
- [x] Require signed commits (optional)
- [x] Restrict who can push/merge
- [x] Enforce linear history (optional)

Protect main Protect your team

6 Pre-commit Hooks (Stop mistakes early)

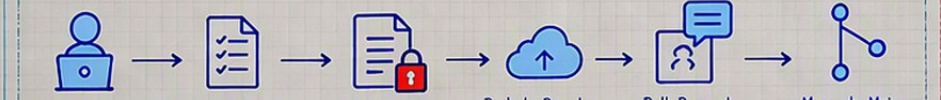
- Pre-commit hooks run automatically before a commit is created.
- Helps catch issues like:
  - Secrets in code (using tools like gitleaks)
  - Large files
  - Code formatting / linting
  - Running tests

Example: Using pre-commit

```
# Install pre-commit
pip install pre-commit
# Create config
pre-commit sample-config > .pre-commit-config.yaml
# Install hooks
pre-commit install
```

Automate checks Save time Avoid leaks

7 Secure Commit Lifecycle (End-to-End Flow)



Key Security Practices:

- [x] Don't commit secrets
- [x] Use .gitignore
- [x] Sign your commits
- [x] Use protected branches
- [x] Enable code reviews & CI/CD
- [x] Use pre-commit hooks
- [x] Regularly audit access

8 Practice / Hands-on

1. Setup .gitignore for a real project
   - Ignore .env, logs, build files, etc.
2. Remove secrets from Git history
   - Use git filter-repo or BFG
   - Rotate the leaked secrets
3. Protect main branch (GitHub/GitLab)
   - Enable PR reviews and status checks
   - Try pushing directly (should be blocked)

Practice makes you confident !

9 Common Commands

Purpose	Command
Check .gitignore	git check-ignore -v <file>
Sign a commit	git commit -S -m "message"
Verify a commit	git log --show-signature
Remove file from history	git filter-repo --path <file> --invert-paths
Check branch protection	(In GitHub/GitLab UI)
List access	(In GitHub/GitLab settings)

Use the right tools for the right problem !

10 Interview Checklist

- [x] What should not be committed to Git?
- [x] Why is .gitignore important?
- [x] How does GPG signing work?
- [x] How do you verify a signed commit?
- [x] What are branch protection rules?
- [x] How do pre-commit hooks help?
- [x] How would you remove a secret from Git history?
- [x] Why is rotation of leaked secrets important?
- [x] How do you manage access control in GitHub?
- [x] Real-world scenario: secret accidentally committed – what steps will you take?

"Secure code today, builds a safer tomorrow !"

Learn | Practice | Implement | Grow